

Course Proposal/Modification

Information needed for College and University Approval

Start Term: Fall 2026

Full course title

Software Engineering Studio I: Founding an Open-Source Project

Abbreviated course title (30 characters max, appears on transcripts)

Software Engineering Studio 1

Catalog number

C S 380

(Proposed. Offered for the first time in Fall 2026 as C S 301R, special topics. First course of the planned Software Engineering Studio sequence, 380 → 480 → 481.)

Justification and summary of request

New course. C S 380 is the first course of a planned three-semester Software Engineering Studio sequence (380 → 480 → 481). It is being taught for the first time in Fall 2026 as C S 301R (special topics); this request seeks a permanent number and catalog listing.

The gap it fills. Our students learn to contribute to software that already exists — a repository, a backlog, and a design are handed to them. They graduate without having done the work that comes before that: finding a real need, proving it is real, arguing for a solution, and standing up a project other people can join. That work is what distinguishes an engineer who can be given a problem from one who can find one, and it is increasingly what industry and graduate programs ask students to show evidence of.

Relationship to the capstone. The capstone currently depends on faculty recruiting external sponsors who “hire” a student team. This course produces student-founded projects and, with them, students who understand a user need deeply enough to act as the project sponsor themselves. Projects that come out of this course may feed the following semester’s capstone, and the student who founded a project can carry it forward as its product owner.

Standalone value. The course is designed to stand on its own for any CS student who wants to learn how to create and run an open-source project. Every student finishes with a launched, documented, contributor-ready project and a public demo; the strongest become capstone seeds.

Assessment emphasis. Grading weights the founding artifacts — discovery, proposal, design, governance, launch — alongside the prototype, so that a strong prototype without a foundation cannot earn top marks. Every checkpoint is scored on a published 100-point analytic rubric.

Credits: 3

Lecture hours per week: 2.5 (two 75-minute sessions)

Lab hours per week: 0

Grading (Standard, Pass/Fail): Standard

Course will be: Taught in person, on campus.

Will this change require new resources (faculty, equipment, space, etc)? No. The course is taught within existing faculty load in a standard classroom. All tooling used is free, and no textbook is required.

Supporting Documents (please attach)

1. Course syllabus (Fall 2026), including grade weights, rubric levels, pass conditions, and policies.
2. Course summary — identity, outcomes, structure, semester map, checkpoints, and assessment.
3. Session-by-session schedule for Fall 2026 (27 sessions plus a finals-week Demo Day).
4. Assignment rubrics — 100-point analytic grids for all twelve checkpoints.
5. Course design document, including the mapping of learning outcomes to the Aims of a BYU Education and to the Software Engineering emphasis's five C's.

Is this course used in other programs and have you conferred with the affected departments and obtained their approval? No. The course is offered by and for Computer Science, and is not currently a requirement in any other program.

When Taught (Fall/Winter/Spring/Summer, Every year, Odd year, Even Year):

Fall, every year.

Enforced Prerequisites

C S 340.

Non-enforced Prerequisites (courses and statements such as “audition required”)

None.

Recommended prerequisites (courses and statements such as “prior programming experience”)

None beyond the enforced prerequisite. Students who have used git on a team project, or who have built and deployed a small full-stack application, will find the build phase more comfortable, but neither is assumed.

Course Description (max 1200 characters, be concise, avoid jargon)

Introduces the practices required to conceive, justify, and found a lasting, open-source software project. Working individually and then in small teams, students discover a real user need, scope a product, write proposal and design documentation, set up a repository with the governance and processes future contributors need, and build an initial working prototype. Topics include product discovery and needfinding, feasibility and scope analysis, software design documents, git-based collaboration, technical communication, open-source licensing and governance, issue and pull-request workflows, and early-stage project management. Emphasizes engineering judgment, technical communication, and building software that serves a real need and that others can join and continue.

Learning Outcomes ([see this CTL resource for important tips](#))

Format is Title: Outcome

Discover and define a real opportunity: Identify a target userbase, investigate their needs through interviews or documented persona and proxy needfinding, and articulate the problem, goals, non-goals, and success criteria.

Evaluate feasibility and scope: Analyze technical risk, breadth, dependencies, and maintainability, and reduce an ambitious vision to a minimum viable product achievable in one semester while preserving a path to grow.

Propose and pitch persuasively: Produce a written project proposal and defend it in an oral pitch that is clear, persuasive, and responsive to critique.

Design a system and a project: Write a design document covering architecture, components, data, interfaces, technology choices, rationale, and risks, and revise it in response to review.

Found an open-source-ready project: Select and justify a software license, and establish repository structure, contribution guidelines, a code of conduct, issue and pull-request templates, onboarding documentation, and governance norms.

Use core git and collaboration workflows: Branch, commit, review, and merge using a maintainer-centered workflow, and write reviewable pull requests and useful commit messages.

Communicate professionally in engineering contexts: Write and respond to issues, bug reports, feature requests, code reviews, and pull requests with precision and professionalism.

Plan, build, and open for contribution: Break work into milestones and issues, build a prototype that proves core viability, and produce a launch and onboarding package with a retrospective that lets new contributors join and the project keep growing.

Course Resources (e.g. textbook)

No textbook purchase is required. All assigned readings are free and linked from the course site.

Readings are drawn from freely available professional sources — the GitHub Open Source Guides, Nielsen Norman Group articles on user interviews and personas, choosealicense.com, and published essays on technical writing and talk design — together with an excerpt of a freely hosted Creative Problem Solving handbook, which is linked rather than distributed.

The course also provides its own handouts and templates: summaries and readings written for the course, interview and ethics guidance grounded in BYU IRB guidance for student projects, persona templates with worked examples, feasibility and learning-plan worksheets, paired strong and weak sample proposals, and the evaluation instruments used in class.

Students need a computer and a GitHub account. All required tooling is free.

Course Format (e.g. lectures, discussion sections, list hours per week for each):

Two 75-minute sessions per week, in person — 2.5 contact hours per week, 0 lab hours.

Session A (Monday, 75 min): short lecture, worked examples, and guided critique.

Session B (Wednesday, 75 min): workshop, lab, peer review, demo, or studio work. Many Wednesdays release early so teams work with the instructor available as a coach.

The course runs as a studio rather than a lecture sequence: most sessions are working sessions, and three are attendance-required assessment events (Pitch Day, the progress demo and repository audit, and the finals-week Demo Day).

Expected total student time is approximately 10 hours per week, including the 2.5 hours in class.

Weekly Schedule of Topics:

Week 1 — Course launch; the founder and maintainer mindset; the creative-problem-solving cycle; divergent problem-space workshop. Students begin an idea journal.

Week 2 — The open-source model, license families, and governance; discovery workshop: interviewing, non-leading questions, honest persona construction, consent and ethics.

Week 3 — Feasibility and risk categories; self-directed learning and the learning plan; convergence workshop — scoring ideas against explicit criteria. Idea Briefs due.

Week 4 — Anatomy of a software proposal; arguing feasibility without overpromising; peer review of proposal scope and risk. Product Definition Brief due.

Week 5 — Pitch workshop and evaluation criteria; Pitch Day: students pitch to persuade and recruit, the room scores every pitch, and team preferences are collected. Written Proposal and Pitch due.

Week 6 — Teams announced and team charters written in class; dissecting a real design document; architecture and data-flow workshop.

Week 7 — Git workflow for maintainers; commit-message critique; hands-on branch, merge, and pull-request lab on the team repository. Design Document and Git Workflow Lab due.

Week 8 — Strong versus weak repository infrastructure; licensing clinic; anatomy of a good issue; workshop to build repository scaffolding and onboarding documentation. Infrastructure Package due.

Week 9 — Turning a design into epics, milestones, and issues; estimation, sequencing, and planning the first vertical slice. Project Plan and Roadmap due.

Week 10 — Evaluating example prototypes and what a first version must prove; a reasoning model for debugging; prototype-plan review and studio build time.

Week 11 — Troubleshooting scope and technical direction; progress demos in small groups with an instructor and peer repository audit and onboarding test. Technical Communication Portfolio and Progress Demo due.

Week 12 — Contributor readiness: what a new contributor needs in order to start. (No class Thanksgiving week.)

Week 13 — Retrospective practice and presentation framing; launch studio — assembling the launch package and testing the on-ramp with a peer. Launch Package and Retrospective due.

Week 14 — Demo dry-run with peer feedback; final studio: repository and document cleanup, and a code freeze before demos.

Finals week — Final Pitch and Demo Day, a three-hour block with capstone faculty invited. Each project answers three questions: what need does this serve, what has been built, and where does it go next and how can others join.

Assessments (e.g. Homework, Labs, Projects):

Twelve graded checkpoints build toward the culminating artifact. The first four are individual; from the sixth week on, work is by team, with solo projects permitted. Each checkpoint is scored on a published 100-point analytic rubric with weighted criteria and four performance levels (Exemplary, Proficient, Developing, Beginning).

Discovery — Idea Briefs and Product Definition Brief (individual): 10%

Written Proposal (individual): 12%

Pitch Presentation (individual): 8%

Design Document (team): 12%

Project Infrastructure Package (team): 12%

Git Workflow Lab and Technical Communication Portfolio: 12%

Project Plan and Roadmap (team): 8%

Prototype — plan, progress demo, and repository audit (team): 16%

Launch Package with Retrospective, and Final Pitch and Demo (team): 10%

Smaller completion-graded deliverables — a team charter, a licensing short response, and a prototype plan — are required and feed the checkpoints they support, but carry no independent weight.

Team accountability. Shared team artifacts receive a team grade adjusted by individual contribution evidence drawn from three streams: git, pull-request, and issue-tracker history; a student contribution log; and confidential peer evaluations at the midpoint and end of term. The individual multiplier defaults to unadjusted and moves only in documented cases, never on peer scores alone.

Pass conditions. To pass, a student must reach at least the Developing level on every rubric criterion of the discovery brief, written proposal, pitch, design document, infrastructure package, prototype, and final presentation. A strong prototype cannot compensate for a missing founding artifact.